

# 存储器

| 2025.5.20 by Xiaopac

## 重点

### 1. cpu 和 存储器的连接

#### 条件

一般会给出可选的存储器：

- N  $K$  \* M 位的 RAM / ROM

CPU 的参数：

- A 根地址线，B 根数据线

以及地址空间。

#### 选择

按照以下的顺序确定要选什么存储器：

- 写出对应的二进制地址
- 确定地址空间的类型：
  - 只读：应该选 ROM
  - 可读可写：应该选 RAM
- 根据 CPU 的数据线数量选择相同位数的存储器。如果手上没有足够位数的存储器，则需要用多片存储器来进行位拓展
- 确定地址空间的大小：根据这个大小来选择对应的存储器（大小  $\leq$  N  $K$ ）。如果手上没有足够大小的存储器，则需要用多片存储器进行字拓展

---

## 2. 存储器的校验

#### 最小距离

又叫汉明距离，可以理解为：

| 在所有合法的编码之间，两个编码不一样的最小差距是多少？

比如：

- 1010101 和 1110101, 只差一个地方 (第2位), 所以距离是 1
- 1010101 和 1111111, 差了 4 位, 所以距离是 4

而“最小距离”指的是: 在你设计的编码系统里, 任何两个合法编码之间, 最少也要差多少位。

- 任意两组合法代码之间 二进制位数的 最少差异
- 编码的纠错、检错能力与编码的最小距离有关

$$L - 1 = D + C \quad (D \geq C)$$

$L$  —— 编码的最小距离 (汉明距离)

$D$  —— 检测错误的位数

$C$  —— 纠正错误的位数

若某编码的  $L = 3$ , 其最大纠错能力?

具有 1 位 纠错能力

## 汉明码

### 求汉明码

为了实现检测功能, 我们需要添加检测位。

$n$  位二进制数的汉明码需增添多少位 ( $k$ ) 检测位?

$$2^k \geq n + k + 1 \quad (\text{汉明不等式})$$

检测位的位置?

| 位置 | 1     | 2     | 3     | 4     | 5     | 6     | 7     |
|----|-------|-------|-------|-------|-------|-------|-------|
|    | $C_1$ | $C_2$ | $b_4$ | $C_4$ | $b_3$ | $b_2$ | $b_1$ |

$$2^i \quad (i = 0, 1, 2, 3, \dots)$$

检测位的取值?

与该位所在的检测“小组”中承担的奇偶校验任务有关

每一个检测位, 会需要负责检查一组位。具体的分工如下:

- $C_1$  检测  $g_1$  小组, 含位 1, 3, 5, 7, 9, 11, ... (XXXX1)
- $C_2$  检测  $g_2$  小组, 含位 2, 3, 6, 7, 10, 11, ... (XXX1X)
- $C_4$  检测  $g_3$  小组, 含位 4, 5, 6, 7, 12, 13, ... (XX1XX)
- $C_8$  检测  $g_4$  小组, 含位 8, 9, 10, 11, 12, 13, 14, 15, 24, ... (X1XXX)

.....

- $g_i$  小组独占第  $2^{i-1}$  位
- $g_i$  和  $g_j$  小组共同占第  $2^{i-1} + 2^{j-1}$  位
- $g_i$ ,  $g_j$  和  $g_l$  小组共同占第  $2^{i-1} + 2^{j-1} + 2^{l-1}$  位

如果按照配偶原则来配置汉明码, 则检测位应使对应小组的 1 的个数为偶数。

### 汉明码的纠错过程

纠错过程本质是利用新的汉明码, 形成新的 检测位

- 计算新的检测位  $P_i$  ( $i=1, 2^1, \dots, 2^{k-1}$ ,  $k$  为检测位位数)

以  $k=3$  为例,  $P_i$  的取值为

$$P_1 = \overset{C_1}{\text{位1}} \oplus \text{位3} \oplus \text{位5} \oplus \text{位7}$$

$$P_2 = \overset{C_2}{\text{位2}} \oplus \text{位3} \oplus \text{位6} \oplus \text{位7}$$

$$P_4 = \overset{C_4}{\text{位4}} \oplus \text{位5} \oplus \text{位6} \oplus \text{位7}$$

对于按 “偶校验” 配置的汉明码

不出错时:  $P_i$  都为 0

根据新的 检测位 的结果, 可以判断出是否出错。同时, 其巧妙之处在于, 如果出错, 我们还可以通过新的检测位, 来判断出出错的位置:

(按偶校验配置) 试问要求传送的信息是什么?

解: 检错过程如下

$$P_1 = \text{位1} \oplus \text{位3} \oplus \text{位5} \oplus \text{位7} = 0$$

$$P_2 = \text{位2} \oplus \text{位3} \oplus \text{位6} \oplus \text{位7} = \mathbf{1}$$

$$P_4 = \text{位4} \oplus \text{位5} \oplus \text{位6} \oplus \text{位7} = \mathbf{1}$$

$$\therefore P_4 P_2 P_1 = 110$$

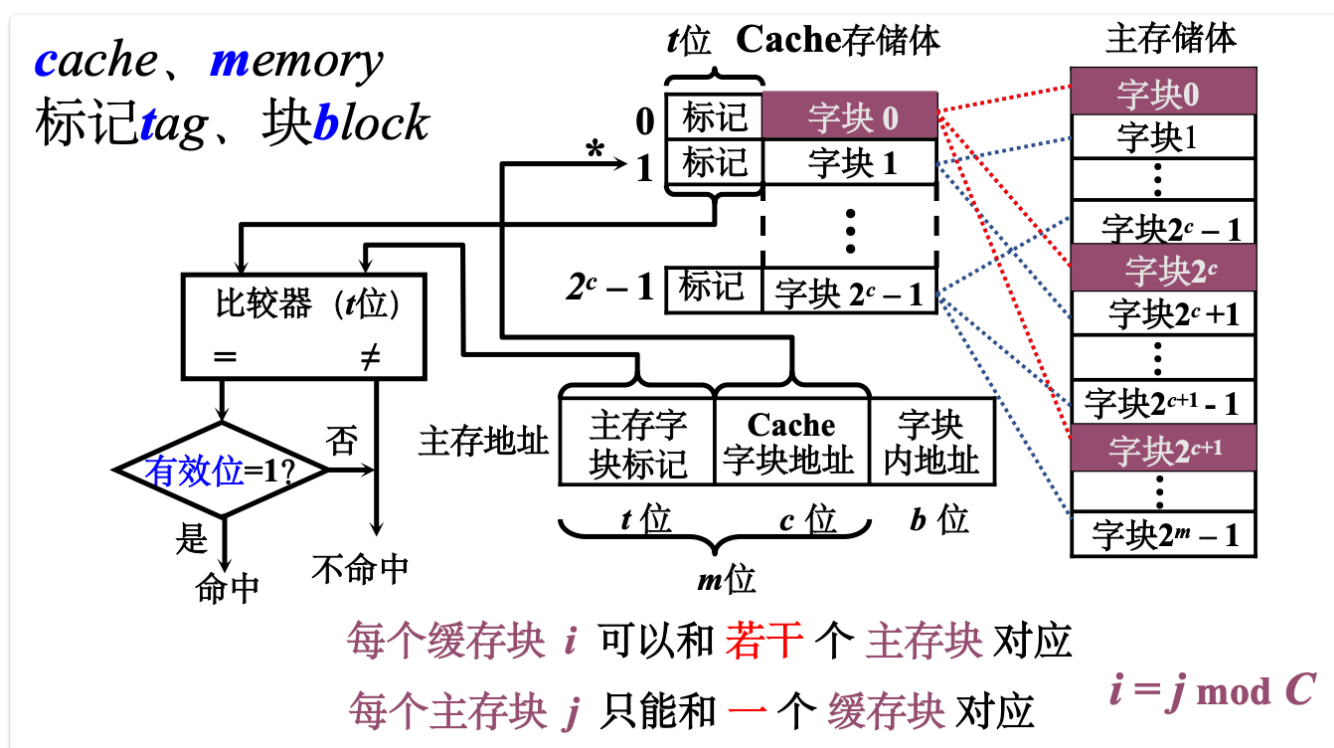
纠错过程: 由 $P_i$ 可知第6位出错, 可纠正为 **0100101**

故要求传送的信息为 **0101**。

注意如果发现是检测位的位置出错, 就不需要纠正。

### 3. Cache 主存映射方式

#### 主存直接映射



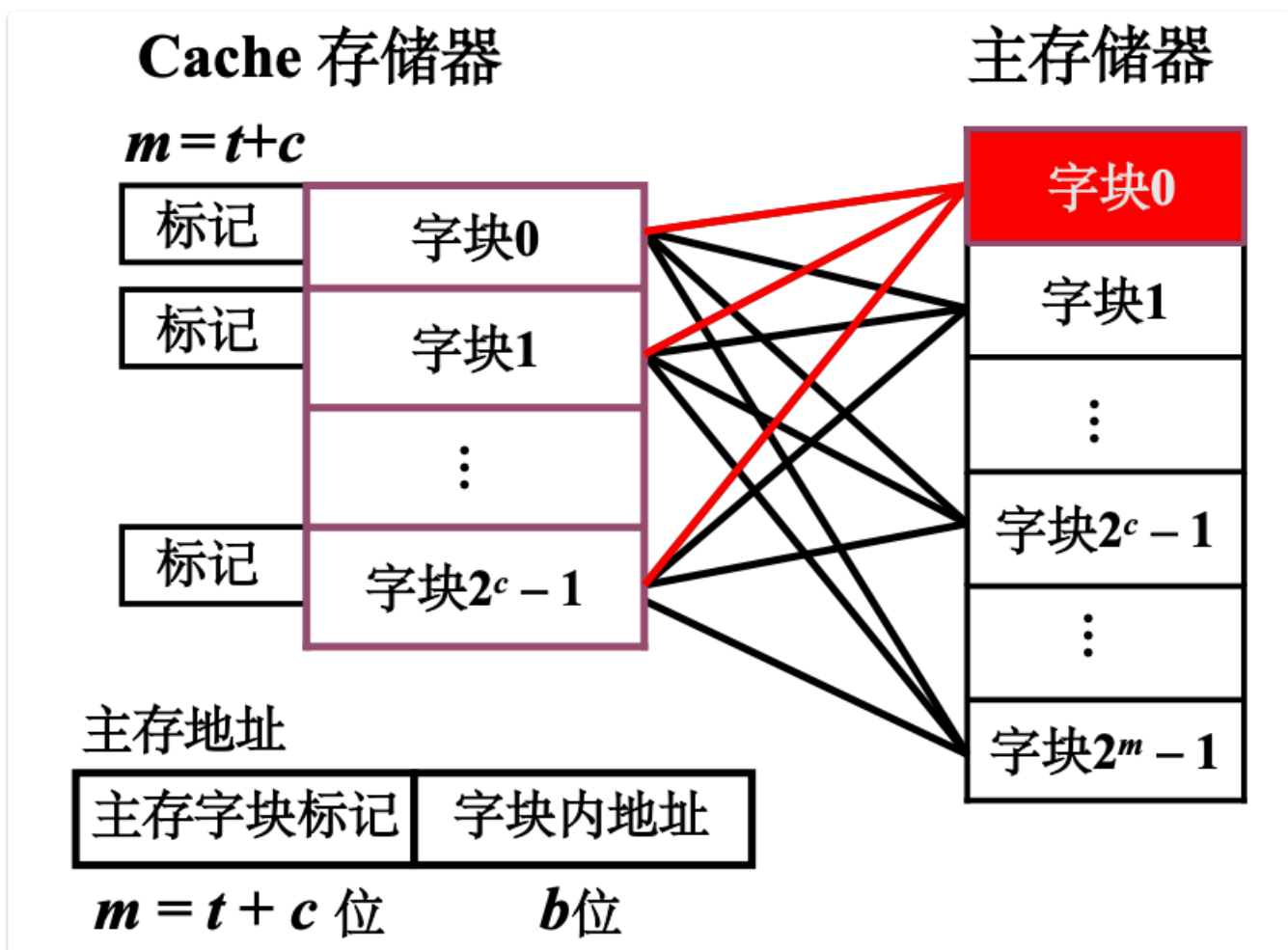
我们把 Cache 存储体, 分为多个 Cache 块。以这里的 0 号 Cache 块为例, 其只可以缓存 **一个** 主存中的 编号为  $2^c$  的倍数的 **字块**。当传来一个 **主存地址**, 按照以下顺序从 cache 里找:

- 分解 **主存地址**, 得到 Cache 字块地址, 即判断为几号 **Cache 块**
- 分解 **主存地址**, 得到 主存字块标记, 与这个 **Cache 块** 的  $t$  位 **标记** 比较
- 相同: hit!

- 不同：miss! 需要重填。

局限：想象一个场景，我们访问的主存地址恰好全部映射在同一个 cache 块，每次读都会发生缺失，同时其他的存储体空闲。

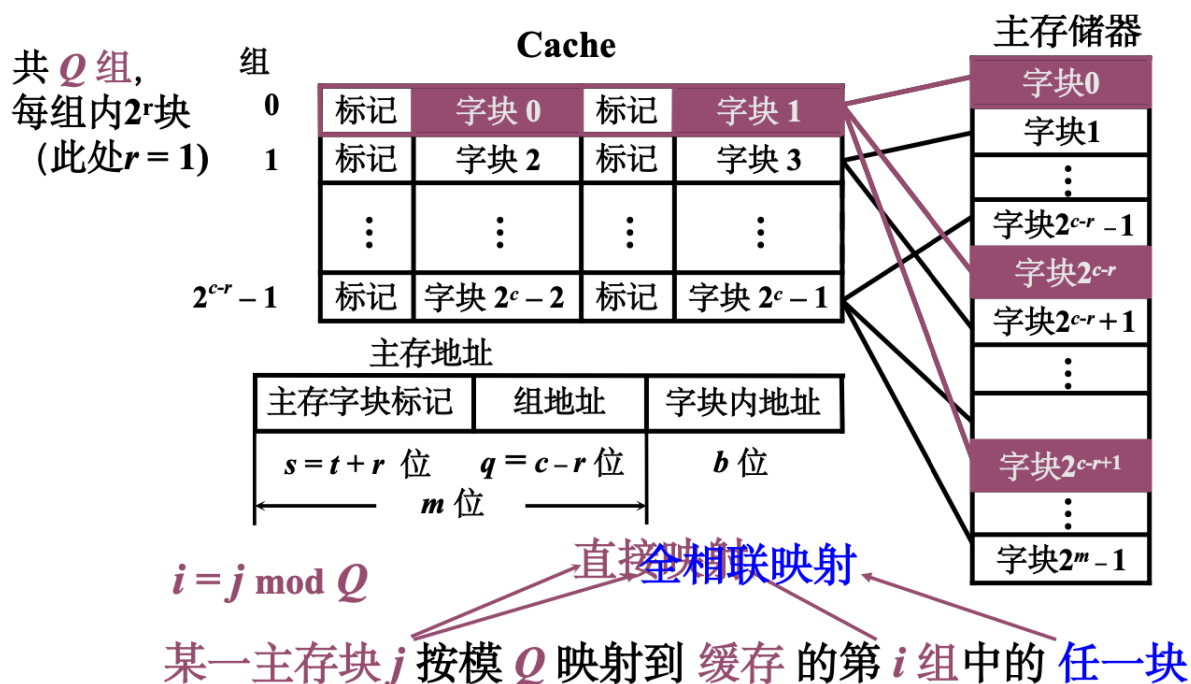
## 主存全相联映射



这种方法更加简单粗暴：所有的字块共用整个存储器。当传来一个 **主存地址**，按照以下顺序从 cache 里找：

- 分解 **主存地址**，得到 **主存字块标记**
- 在整个 Cache 存储器的所有 **标记** 里找这个 **主存字块标记**
- 找到：hit!
- 找不到：miss! 需要重填（填充/使用替换策略）

## 主存组相联映射



这种映射方法，可以看作是 主存全相联映射 的改进。以这里的 0 号 Cache 块为例，现在可以缓存 **多个** 主存中的 编号为  $2^c$  的倍数的 **字块**。

对比：（主存全相联映射）以这里的 0 号 Cache 块为例，其只可以缓存 **一个** 主存中的 编号为  $2^c$  的倍数的 **字块**。

当传来一个 **主存地址**，按照以下顺序从 cache 里找：

- 分解 **主存地址**，得到 **组地址**，即判断为几号 **Cache 组**
- 分解 **主存地址**，得到 **主存字块标记**，在这个 **Cache 组** 的多个 **标记** 里寻找，是否存在匹配的 Cache 块。
- 找到：hit!
- 找不到：miss! 需要重填。

summary

- 主存地址划分如下表，其中：块内地址相同
- Cache块 (line) 地址  $\approx$  Cache组地址
- 通过主存块标记与Cache块标记的内容比对判断是否命中
- Cache块标记是**Cache内容的一部分**
- 有Cache时，主存储器的结构会变得复杂

| 映射方式  | t位    | r位       | c-r位 | b位   | 备注                             |
|-------|-------|----------|------|------|--------------------------------|
| 全相联映射 | 主存块标记 |          |      | 块内地址 | 主存块 $\rightarrow$ Cache任一块     |
| 直接映射  | 主存块标记 | Cache块地址 |      | 块内地址 | 模 $2^c$ ，标记t位                  |
| 组相联映射 | 主存块标记 |          | 组地址  | 块内地址 | 每组 $2^r$ 路,模 $2^{c-r}$ ,标记t+r位 |